Project Workflow and Replication Notes

Updated: 2026-05-31

This note documents the current practical workflow used to modernize, typeset, audit, translate, package, and publish public-domain mathematical and physical manuscripts. It is intentionally operational: the aim is that another reader can reproduce the pipeline with their own scans, their own compute, and their own publication account.

Project Aim

The project aims to turn scanned historical mathematical manuscripts into modern, source-checkable LaTeX editions, then into translations where useful. The public archive is organized so that a reader can download a readable PDF directly, while TeX sources, original scans, provenance, and audit material remain available in accompanying ZIP artifacts.

The preferred public shape is:

· one reader-facing PDF per work or coherent volume;
· a paired translation PDF when translation exists;
· a ZIP containing TeX, source scans, intermediate files, manifests, and audit notes;
· a stable all-versions Zenodo link for long-term citation;
· a GitHub mirror for forkable TeX and issue-style collaboration.

Current AI-Assisted Workflow

1. ChatGPT web project sessions help identify useful public-domain works, plan batches, and perform focused translation or repair passes.
2. Codex downloads and indexes public scans, builds local inventories, writes scripts, packages releases, audits public files, publishes through the Zenodo API, and mirrors clean material to GitHub.
3. Kimi K2.6 agent swarms have been used for large first-pass transcription into TeX, especially where hundreds of pages can be split into small source-checkable units.
4. ChatGPT Pro sessions check, translate, repair, and recompile TeX. In practice, several extended sessions can run in parallel against different works.
5. Claude Code has been useful as a local coding and LaTeX-repair worker, especially for direct scan-based chunk repair, EGA translation drafts, and large TeX cleanup triage.
6. The publication pass promotes the cleanest available reader PDFs while preserving rougher drafts and provenance inside ZIP artifacts rather than pretending they are final.

This is a work-in-progress research and preservation workflow, not a benchmark. The exact model mix changes as rate limits and availability change.

Observed Cost and Throughput Notes

These figures are informal project notes, not controlled measurements.

· A Claude Max weekly allowance was consumed on major Cayley repair work plus partial EGA translation. That produced double-digit-percent progress on a very large Cayley corpus and meaningful EGA additions, but at a high token cost.
· Codex carried the Zenodo/GitHub/indexing/publication pipeline on a separate weekly allowance while web sessions continued translation and repair work.
· ChatGPT Pro web sessions were useful in parallel: several extended sessions could translate or repair a few to several pages per 30-60 minutes each, depending on mathematical density and whether source TeX was already available.
· Kimi K2.6 agent swarms were effective for broad first-pass TeX transcription, but monthly credit exhaustion became a practical scheduling constraint.
· If significant paid compute were available, the bottleneck would move from raw transcription to audit discipline: page completeness, formula fidelity, typography, public naming, and provenance.

Local Tool Stack

Core tools:

· Windows PowerShell for orchestration, file inventory, chunking, and local process control.
· Python 3 with PyMuPDF (`fitz`) for PDF inspection, page counts, text extraction, rendering checks, redaction/surface cleanup, and simple generated PDFs.
· Python `pdf2image` plus Poppler-compatible rendering for spot-check PNGs from generated PDFs and long audit packets.
· Python standard-library `zipfile`, `hashlib`, `json`, `csv`, `pathlib`, and `subprocess` for packaging and manifests.
· Pandoc for Markdown-to-DOCX/PDF conversion of proposal, workflow, and prompt-provenance packets.
· LibreOffice (`soffice`, installed via `winget` as TheDocumentFoundation.LibreOffice) for DOCX/PDF conversion and document render QA where available.
· MiKTeX / XeLaTeX / pdfLaTeX / LuaLaTeX for TeX compilation.
· Git and GitHub over SSH for the public mirror.
· Zenodo REST API for versioned archival deposits.

OCR and formula witnesses tested or installed:

· Docling and RapidOCR are currently useful for rough Latin-script/French mathematical page witnesses. They preserve enough prose to help a repair pass, but spacing, symbols, formulas, and non-Latin scripts still need source checking.
· pix2tex / LaTeX-OCR is installed and runs locally on CPU. It is useful only for tightly cropped isolated formulas; full pages and mixed text/formula paragraphs produced unusable output in smoke tests.
· Surya OCR is installed in a formula/OCR environment, but the current v2 command path is blocked on this workstation until a Docker/vLLM or llama.cpp server backend is configured.
· Pix2Text and olmOCR are next candidates to test for mixed text/math/table extraction.
· PyTorch with CUDA should be used where GPU acceleration is available.

These OCR tools are helpful witnesses, but none should be treated as authoritative for historical mathematical text. The reliable workflow is still scan -> OCR/formula witness -> TeX candidate -> compile -> visual/text audit -> source check.

Repeatable Release Procedure

1. Create a stable local project tree:
   · `sources/` for original scans and downloaded references;
   · `work/` for OCR/transcription/translation sessions;
   · `release_candidates/` for public staging;
   · `tools/` for repeatable scripts;
   · `reports/` for audits and manifests.
2. Index everything, including ZIP/RAR contents, with hashes, sizes, page counts, and inferred author/work metadata.
3. Split large scans into practical chunks for OCR or model sessions. Keep chunk names boring and source-checkable: author, volume, printed page range.
4. Produce TeX and compile early. A compiled PDF, even imperfect, exposes missing fonts, overfull boxes, formula markup leaks, and broken page structure.
5. Audit before promotion:
   · page count and file size sanity;
   · visible TeX commands in PDF text;
   · process notes or assistant-facing text in public PDFs;
   · local paths or personal names;
   · unreadable squares/missing glyphs;
   · obvious font-size or page-layout failures;
   · source completeness against available scans.

6. Promote only the cleanest current reader as top-level public material. Preserve older drafts, raw drops, source scans, TeX, logs, and manifests inside ZIP artifacts.
7. Publish through Zenodo as a new version of an existing logical record whenever possible. Keep stable all-version DOI links in GitHub and descriptions.
8. Mirror the public TeX/readers/manifests to GitHub for collaboration, issues, forks, and easier browsing.
9. Keep a to-do list per author/work: missing page ranges, known typography defects, translation status, source-scan status, and audit confidence.

Public Naming Rules

Public names should describe the work, not the internal batch that produced it. Good names look like:

· `Emmy Noether - Paper 11 - Equations with Prescribed Group - English Translation.pdf`
· `SGA 5 - High-Fidelity Working Translation through Expose VII.pdf`
· `Cayley - Collected Mathematical Papers, Volume X - Source-Checked Modern LaTeX Slice Reader.pdf`

Avoid naming public files after temporary model drops, repair passes, chat sessions, or internal batch numbers. Put that detail in manifests and artifact ZIPs when provenance is needed.

Quality Tiers

· Reader-facing: clean enough to be useful directly; no assistant notes; coherent names; readable typography.
· Source-checkable working draft: useful but not final; should still include source material and caveats.
· Artifact/provenance: raw outputs, partial drafts, logs, and superseded versions preserved for audit and recovery.
· Held candidate: not promoted because it is unreadable, incomplete, has visible internal markup, or is worse than the current public reader.

What To Hand To Another Local Agent

Give the agent:

· the current reader PDF;
· TeX sources;
· source scans or page images;
· a manifest mapping public pages to source pages;
· the known-defects report;
· the local style rules and naming rules;
· a small compile command or build script;
· instructions to return a cumulative replacement, not only a delta.

This packet includes sanitized copies of the scripts and templates that implement that pattern.